Transformers for Vision

Vision Transformer Encoder and Patch Embeddings

The Challenge: How to Apply Transformers to Images?

Transformers were designed to process 1D sequences of tokens (e.g., text). However, images are inherently 2D grids of pixels.

The Problem

Applying self-attention directly to every pixel in an image is computationally infeasible. For a modest 224x224 image, the number of pixel-to-pixel interactions would be $(224^2)^2 \approx 2.5 \times 10^9$.

The Solution: Treat Images as a Sequence of Patches

The core idea of the Vision Transformer (ViT) is to break an image down into a series of smaller, fixed-size patches and treat these patches as the tokens in a sequence.

Introduction to the Vision Transformer (ViT)

A Different Approach: The Transformer

The **Transformer** architecture achieved revolutionary success in Natural Language Processing (NLP) by processing the relationships between all words in a sentence simultaneously, giving it a **global understanding** of the context.

Introduction to the Vision Transformer (ViT)

A Different Approach: The Transformer

The **Transformer** architecture achieved revolutionary success in Natural Language Processing (NLP) by processing the relationships between all words in a sentence simultaneously, giving it a **global understanding** of the context.

Key Details

- **Model Architecture:** It is an encoder-only Transformer.

Introduction to the Vision Transformer (ViT)

A Different Approach: The Transformer

The **Transformer** architecture achieved revolutionary success in Natural Language Processing (NLP) by processing the relationships between all words in a sentence simultaneously, giving it a **global understanding** of the context.

Key Details

- **Model Architecture:** It is an encoder-only Transformer.
- **Core Innovation:** It breaks an image down into a series of fixed-size **patches**, like pieces of a jigsaw puzzle.

Introduction to the Vision Transformer (ViT)

A Different Approach: The Transformer

The **Transformer** architecture achieved revolutionary success in Natural Language Processing (NLP) by processing the relationships between all words in a sentence simultaneously, giving it a **global understanding** of the context.

Key Details

- **Model Architecture:** It is an encoder-only Transformer.
- **Core Innovation:** It breaks an image down into a series of fixed-size **patches**, like pieces of a jigsaw puzzle.
- **Applications:** Utilized mainly for learning image representations.

Introduction to the Vision Transformer (ViT)

A Different Approach: The Transformer

The **Transformer** architecture achieved revolutionary success in Natural Language Processing (NLP) by processing the relationships between all words in a sentence simultaneously, giving it a **global understanding** of the context.

Key Details

- **Model Architecture:** It is an encoder-only Transformer.
- **Core Innovation:** It breaks an image down into a series of fixed-size **patches**, like pieces of a jigsaw puzzle.
- **Applications:** Utilized mainly for learning image representations.
- **Advantage:** Utilizes the State-Of-The-Art Attention mechanism for vision modelling

Patch Embeddings: From Pixels to Tokens

Core innovation: Conversion of 2D image into a 1D sequence of vectors (or "tokens") that a standard Transformer can understand. It is a three-step process:

Patch Embeddings: From Pixels to Tokens

Core innovation: Conversion of 2D image into a 1D sequence of vectors (or "tokens") that a standard Transformer can understand. It is a three-step process:

1. **Patching:** The input image (e.g., 224x224 pixels) is split into a grid of smaller, non-overlapping patches (e.g., 16x16 pixels each).

Patch Embeddings: From Pixels to Tokens

Core innovation: Conversion of 2D image into a 1D sequence of vectors (or "tokens") that a standard Transformer can understand. It is a three-step process:

1. **Patching:** The input image (e.g., 224x224 pixels) is split into a grid of smaller, non-overlapping patches (e.g., 16x16 pixels each).
2. **Flattening:** Each 2D patch is unrolled or "flattened" into a single long vector. For a 16x16 RGB patch, this vector would have $16 \times 16 \times 3 = 768$ dimensions.

Patch Embeddings: From Pixels to Tokens

Core innovation: Conversion of 2D image into a 1D sequence of vectors (or "tokens") that a standard Transformer can understand. It is a three-step process:

1. **Patching:** The input image (e.g., 224x224 pixels) is split into a grid of smaller, non-overlapping patches (e.g., 16x16 pixels each).
2. **Flattening:** Each 2D patch is unrolled or "flattened" into a single long vector. For a 16x16 RGB patch, this vector would have $16 \times 16 \times 3 = 768$ dimensions.
3. **Linear Projection:** This long, raw pixel vector is then fed through a standard MLP layer to project it down to the model's desired embedding dimension, d.

Patch Embeddings: From Pixels to Tokens

Core innovation: Conversion of 2D image into a 1D sequence of vectors (or "tokens") that a standard Transformer can understand. It is a three-step process:

1. **Patching:** The input image (e.g., 224x224 pixels) is split into a grid of smaller, non-overlapping patches (e.g., 16x16 pixels each).
2. **Flattening:** Each 2D patch is unrolled or "flattened" into a single long vector. For a 16x16 RGB patch, this vector would have $16 \times 16 \times 3 = 768$ dimensions.
3. **Linear Projection:** This long, raw pixel vector is then fed through a standard MLP layer to project it down to the model's desired embedding dimension, d.

The output of this process is the **patch embedding**(analogous to word embedding in text processing), a vector that represents the semantic content of a small region of the image.

Patch Embeddings: The Convolutional Shortcut

Instead of manually splitting, flattening, and projecting patches, the entire patch embedding process can be efficiently implemented as a **single 2D convolution layer**.

Patch Embeddings: The Convolutional Shortcut

Instead of manually splitting, flattening, and projecting patches, the entire patch embedding process can be efficiently implemented as a **single 2D convolution layer**.

The "Trick"

A ‘Conv2d‘ layer is configured with specific parameters:

Patch Embeddings: The Convolutional Shortcut

Instead of manually splitting, flattening, and projecting patches, the entire patch embedding process can be efficiently implemented as a **single 2D convolution layer**.

The "Trick"

A ‘Conv2d‘ layer is configured with specific parameters:

- **Kernel Size = Patch Size**
  (e.g., 16×16)

Patch Embeddings: The Convolutional Shortcut

Instead of manually splitting, flattening, and projecting patches, the entire patch embedding process can be efficiently implemented as a **single 2D convolution layer**.

The "Trick"

A ‘Conv2d‘ layer is configured with specific parameters:

- **Kernel Size = Patch Size**
  (e.g., 16×16)
- **Stride = Patch Size**
  (e.g., 16)

Patch Embeddings: The Convolutional Shortcut

Instead of manually splitting, flattening, and projecting patches, the entire patch embedding process can be efficiently implemented as a **single 2D convolution layer**.

The "Trick"

A ‘Conv2d‘ layer is configured with specific parameters:

- **Kernel Size = Patch Size**
  (e.g., 16×16)
- **Stride = Patch Size**
  (e.g., 16)
- **Output Channels = Embedding Dim**
  (e.g., 768)

Patch Embeddings: The Convolutional Shortcut

Instead of manually splitting, flattening, and projecting patches, the entire patch embedding process can be efficiently implemented as a **single 2D convolution layer**.

The "Trick"

A ‘Conv2d‘ layer is configured with specific parameters:

- **Kernel Size = Patch Size**
  (e.g., 16×16)
- **Stride = Patch Size**
  (e.g., 16)
- **Output Channels = Embedding Dim**
  (e.g., 768)

This setup makes the convolution behave exactly like a sliding, non-overlapping patch extractor and projector in one step.

The [CLS] Token

Inspired by the BERT model in NLP, the Vision Transformer prepends a special, learnable token to the sequence, commonly called the '[CLS]' (classification) token.

The [CLS] Token

Inspired by the BERT model in NLP, the Vision Transformer prepends a special, learnable token to the sequence, commonly called the '[CLS]' (classification) token.

What is its Purpose?

1. This token acts as a **global information aggregator**.

The [CLS] Token

Inspired by the BERT model in NLP, the Vision Transformer prepends a special, learnable token to the sequence, commonly called the '[CLS]' (classification) token.

What is its Purpose?

1. This token acts as a **global information aggregator**.
2. It is not linked to specific patch features.

The [CLS] Token

Inspired by the BERT model in NLP, the Vision Transformer prepends a special, learnable token to the sequence, commonly called the '[CLS]' (classification) token.

What is its Purpose?

1. This token acts as a **global information aggregator**.
2. It is not linked to specific patch features.
3. It gathers and summarizes information from all the patch embeddings.

The [CLS] Token

Inspired by the BERT model in NLP, the Vision Transformer prepends a special, learnable token to the sequence, commonly called the '[CLS]' (classification) token.

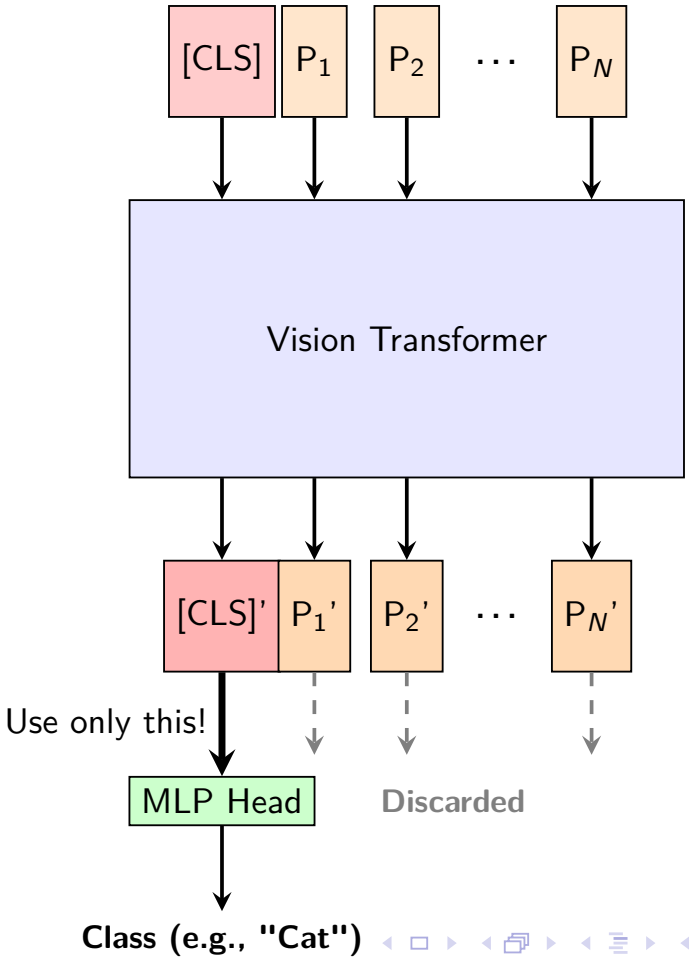
What is its Purpose?

1. This token acts as a **global information aggregator**.
2. It is not linked to specific patch features.
3. It gathers and summarizes information from all the patch embeddings.
4. It is highly useful for classification and related vision tasks.

The [CLS] token - Application

How it Works:

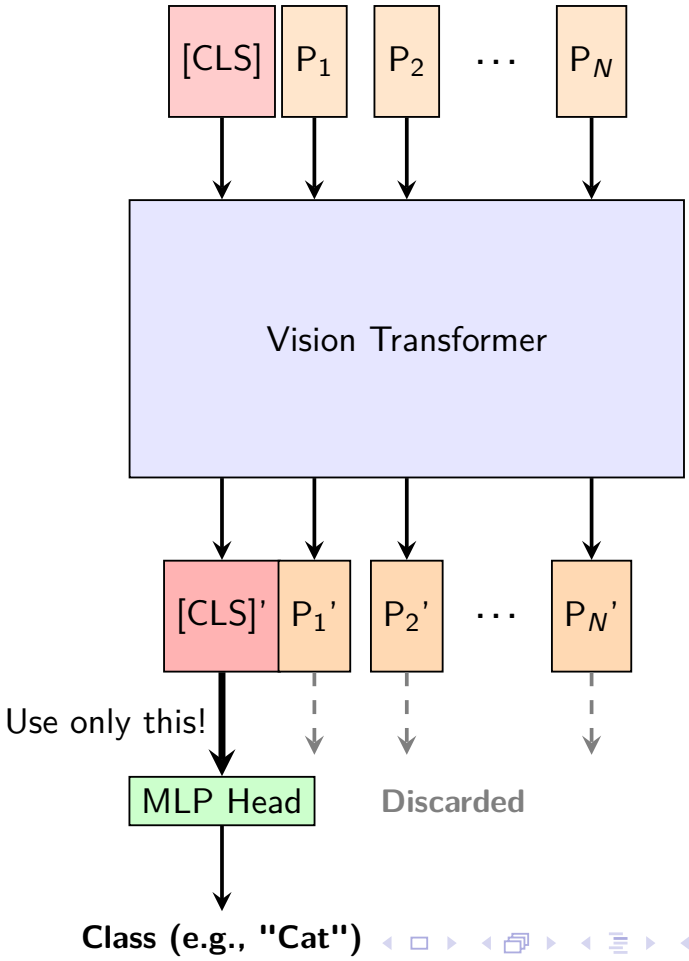
1. A learnable '[CLS]' vector is **prepended** to the sequence of patch embeddings.



The [CLS] token - Application

How it Works:

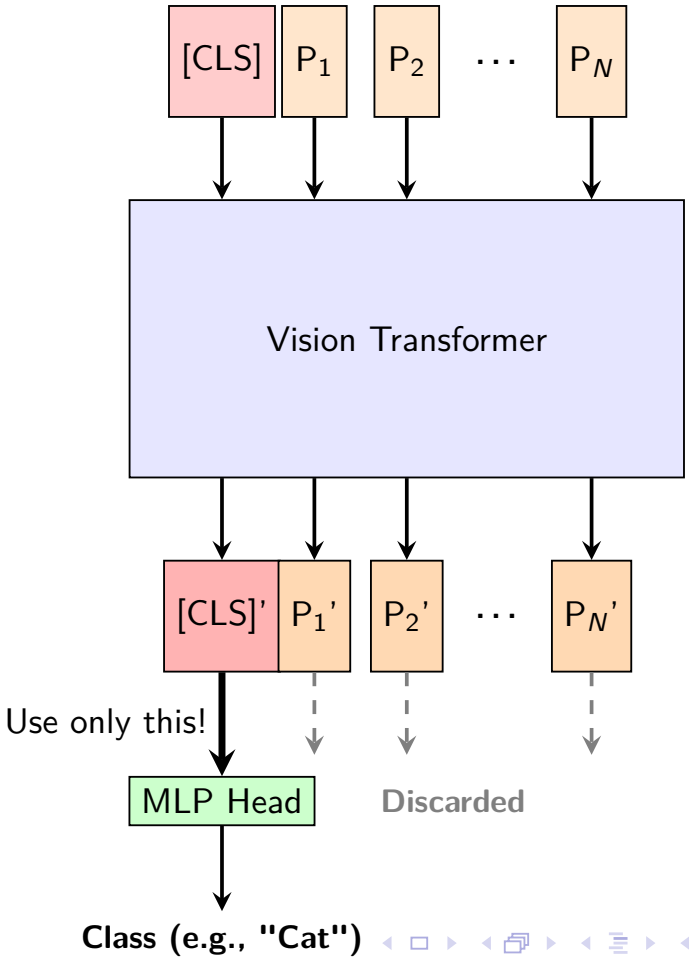
1. A learnable '[CLS]' vector is **prepended** to the sequence of patch embeddings.
2. The entire sequence is fed through the Vision Transformer.



The [CLS] token - Application

How it Works:

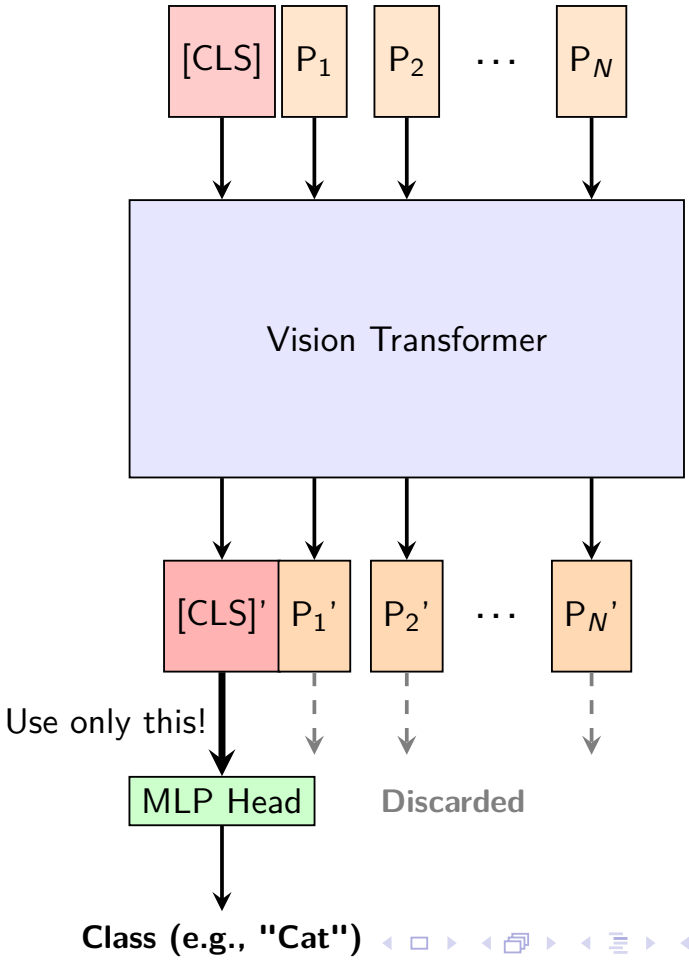
1. A learnable '[CLS]' vector is **prepended** to the sequence of patch embeddings.
2. The entire sequence is fed through the Vision Transformer.
3. The final, output vector corresponding to the '[CLS]' token's position is considered the representation of the **entire image**.



The [CLS] token - Application

How it Works:

1. A learnable '[CLS]' vector is **prepended** to the sequence of patch embeddings.
2. The entire sequence is fed through the Vision Transformer.
3. The final, output vector corresponding to the '[CLS]' token's position is considered the representation of the **entire image**.
4. The final [CLS] vector is then passed to a simple MLP head for the final classification.



Task: Converting Image to Patch Embeddings

Let's consider a practical scenario for creating patch embeddings.

Task: Converting Image to Patch Embeddings

Let's consider a practical scenario for creating patch embeddings.

The Task Parameters

- **Input Image Size:** $H \times W = 224 \times 224$ pixels (assuming 3 color channels).

Task: Converting Image to Patch Embeddings

Let's consider a practical scenario for creating patch embeddings.

The Task Parameters

- **Input Image Size:** $H \times W = 224 \times 224$ pixels (assuming 3 color channels).
- **Patch Size:** $P \times P = 16 \times 16$ pixels.

Task: Converting Image to Patch Embeddings

Let's consider a practical scenario for creating patch embeddings.

The Task Parameters

- **Input Image Size:** $H \times W = 224 \times 224$ pixels (assuming 3 color channels).
- **Patch Size:** $P \times P = 16 \times 16$ pixels.
- **Embedding Dimension:** $D = 768$. This is the desired feature dimension for each patch.

Task: Converting Image to Patch Embeddings

Let's consider a practical scenario for creating patch embeddings.

The Task Parameters

- **Input Image Size:** $H \times W = 224 \times 224$ pixels (assuming 3 color channels).
- **Patch Size:** $P \times P = 16 \times 16$ pixels.
- **Embedding Dimension:** $D = 768$. This is the desired feature dimension for each patch.

Calculating the Number of Patches

The number of patches, N, is determined by dividing the image dimensions by the patch size.

$$N = \left(\frac{H}{P}\right) \times \left(\frac{W}{P}\right) = \left(\frac{224}{16}\right) \times \left(\frac{224}{16}\right) = 14 \times 14 = 196$$

Converting Image to Patch Embeddings

Calculating the Number of Patches

The number of patches, N, is determined by dividing the image dimensions by the patch size.

$$N = \left(\frac{H}{P}\right) \times \left(\frac{W}{P}\right) = \left(\frac{224}{16}\right) \times \left(\frac{224}{16}\right) = 14 \times 14 = 196$$

- This means we will get **196** patch embeddings.
- Each patch embedding will be a vector of **768** dimensions.
- These **196** tokens (plus a '[class]' token if used) will form the sequence input to the Transformer encoder.

The final sequence shape will be $\mathbb{R}^{(\text{num_patches},\text{embedding_dim})} = \mathbb{R}^{(196,768)}$.

Patch Embeddings: Convolutional Illustration

Converting Image to Sequence (Practical Implementation)

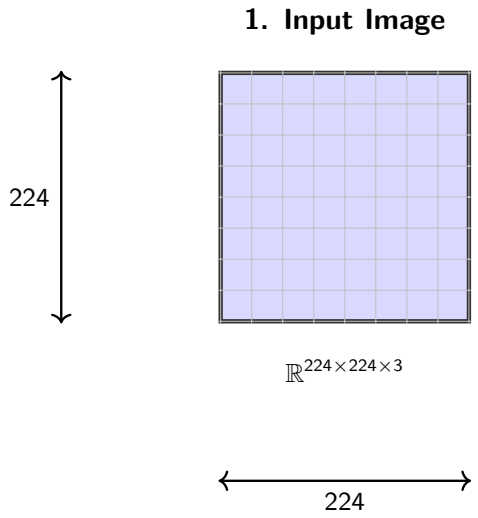


Figure: Convolutional implementation of patch embeddings in Vision Transformers. The input image is divided into non-overlapping patches using a strided convolution with learnable filters.

Patch Embeddings: Convolutional Illustration

Converting Image to Sequence (Practical Implementation)

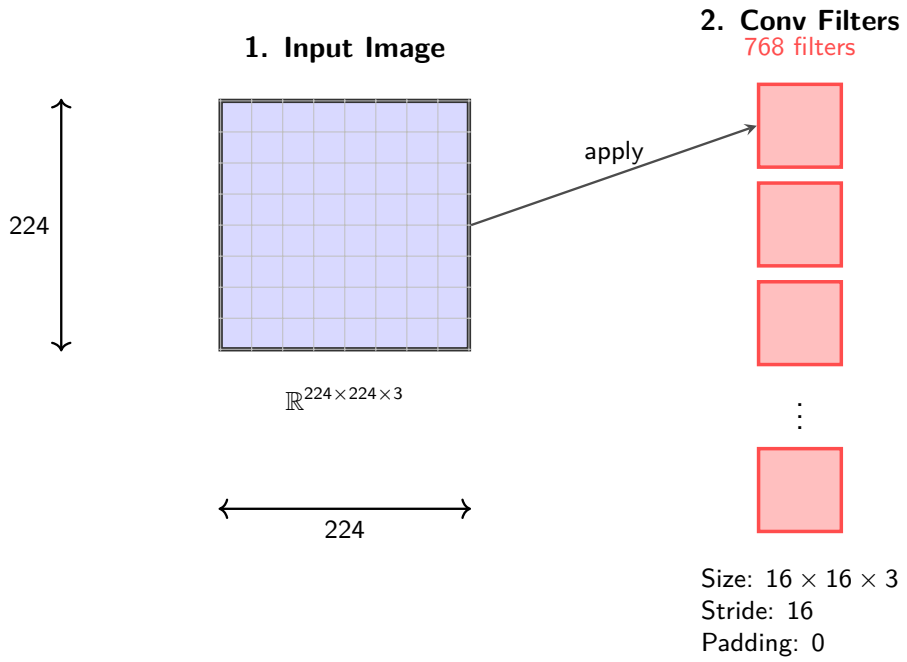


Figure: Convolutional implementation of patch embeddings in Vision Transformers. The input image is divided into non-overlapping patches using a strided convolution with learnable filters.

Patch Embeddings: Convolutional Illustration

Converting Image to Sequence (Practical Implementation)

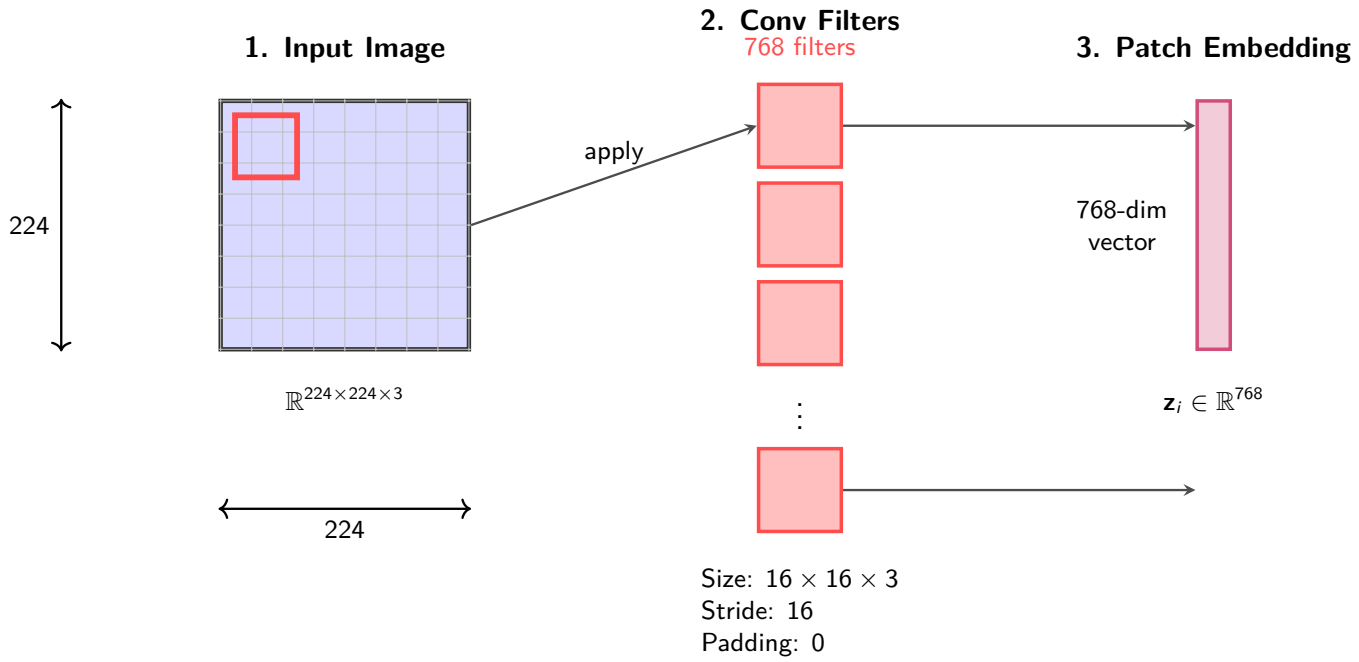


Figure: Convolutional implementation of patch embeddings in Vision Transformers. The input image is divided into non-overlapping patches using a strided convolution with learnable filters.

Patch Embeddings: Convolutional Illustration

Converting Image to Sequence (Practical Implementation)

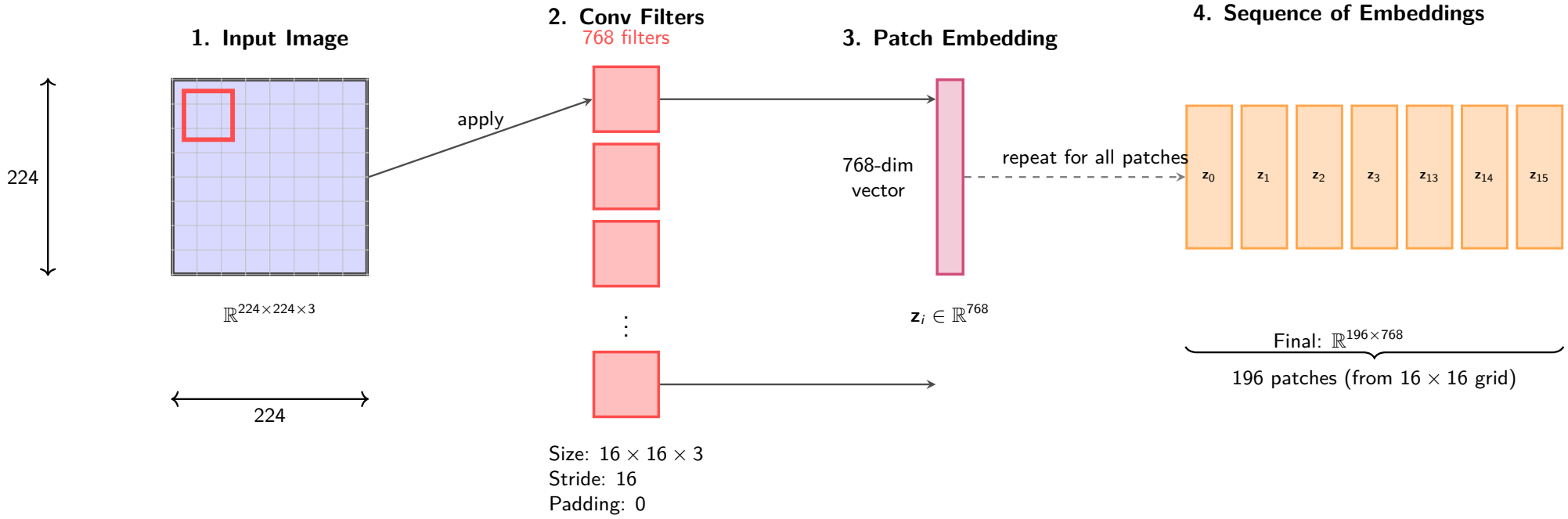


Figure: Convolutional implementation of patch embeddings in Vision Transformers. The input image is divided into non-overlapping patches using a strided convolution with learnable filters.

The ViT Architecture

Once the input sequence of patch embeddings is prepared, the rest of the model architecture is similar to Transformer encoder.

The ViT Architecture

Once the input sequence of patch embeddings is prepared, the rest of the model architecture is similar to Transformer encoder.

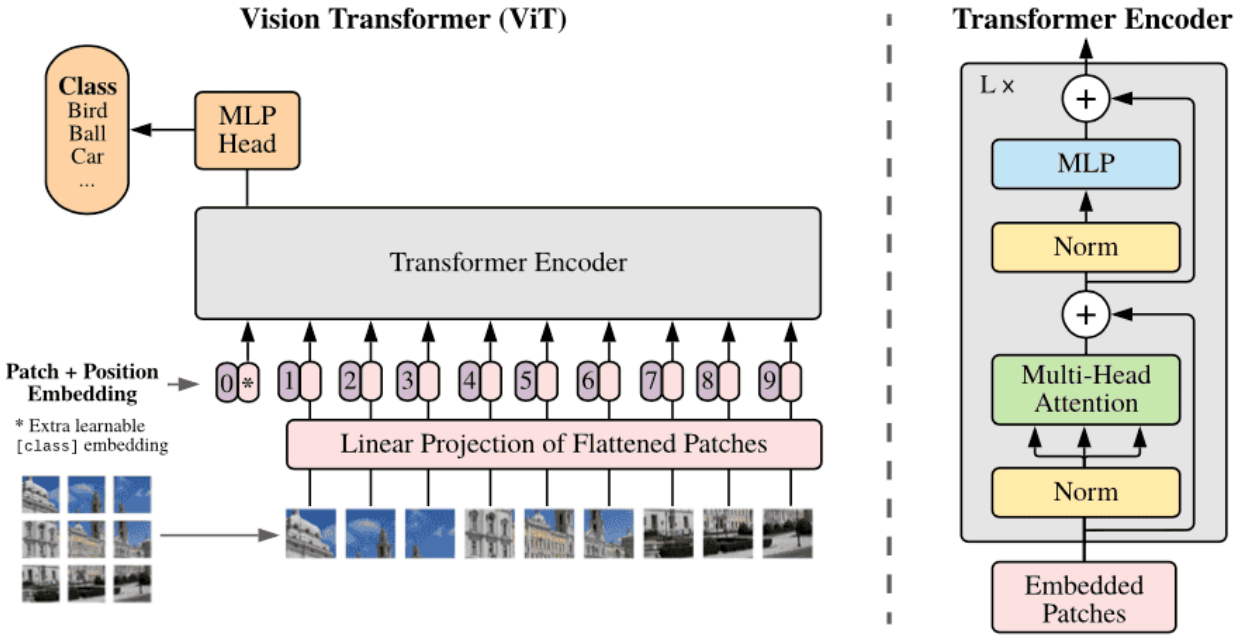


Figure: ViT Architecture. (source: "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale")

The Complete ViT Data Pipeline

1. **Image to Patch Embeddings via Convolution:** A **2D convolution layer** is configured with specific parameters to act as a patch embedder:
   - The convolution's **kernel size** is set to the patch size (e.g., 16×16).
   - The **stride** is also set to the patch size (e.g., 16) to ensure the "patches" are non-overlapping.
   - The number of **output channels** is set to the desired embedding dimension (d, e.g., 768).

   The output of this single operation is a feature map that is then flattened into the final sequence of N patch embeddings.

The Complete ViT Data Pipeline

1. **Image to Patch Embeddings via Convolution:** A **2D convolution layer** is configured with specific parameters to act as a patch embedder:
   - The convolution's **kernel size** is set to the patch size (e.g., 16×16).
   - The **stride** is also set to the patch size (e.g., 16) to ensure the "patches" are non-overlapping.
   - The number of **output channels** is set to the desired embedding dimension (d, e.g., 768).

   The output of this single operation is a feature map that is then flattened into the final sequence of N patch embeddings.

2. **Prepend '[CLS]' Token Embedding:** A special, learnable **'[CLS]' token embedding** ($\mathbf{z}_0$) is added at the beginning of the sequence of patch embeddings. This token will aggregate global image information for classification.

The Complete ViT Data Pipeline

1. **Image to Patch Embeddings via Convolution:** A **2D convolution layer** is configured with specific parameters to act as a patch embedder:
   - The convolution's **kernel size** is set to the patch size (e.g., 16×16).
   - The **stride** is also set to the patch size (e.g., 16) to ensure the "patches" are non-overlapping.
   - The number of **output channels** is set to the desired embedding dimension (d, e.g., 768).

   The output of this single operation is a feature map that is then flattened into the final sequence of N patch embeddings.

2. **Prepend '[CLS]' Token Embedding:** A special, learnable **'[CLS]' token embedding** ($\mathbf{z}_0$) is added at the beginning of the sequence of patch embeddings. This token will aggregate global image information for classification.

3. **Add Positional Embeddings:** To imbue the sequence with spatial order information (which self-attention otherwise lacks), a set of learnable **positional embeddings** ($\mathbf{P}_{pos}$) are added to each token embedding (both '[CLS]' and patch embeddings).

$$\mathbf{e}_i = \mathbf{z}_i + \mathbf{P}_{pos,i}$$

The Complete ViT Data Pipeline

4. **Transformer Encoder Layers:** This final sequence of enriched embeddings (including the '[CLS]' token) is then fed into a stack of standard Transformer Encoder layers. Each layer uses Multi-Head Self-Attention to learn relationships between all tokens.

The Complete ViT Data Pipeline

4. **Transformer Encoder Layers:** This final sequence of enriched embeddings (including the '[CLS]' token) is then fed into a stack of standard Transformer Encoder layers. Each layer uses Multi-Head Self-Attention to learn relationships between all tokens.
5. **MLP Head for Classification:** After passing through the encoder, the output vector corresponding to the '[CLS]' token's position is extracted. This vector, representing the entire image, is then passed through a simple Multi-Layer Perceptron (MLP) for the final image classification prediction.

The Complete ViT Data Pipeline

4. **Transformer Encoder Layers:** This final sequence of enriched embeddings (including the '[CLS]' token) is then fed into a stack of standard Transformer Encoder layers. Each layer uses Multi-Head Self-Attention to learn relationships between all tokens.
5. **MLP Head for Classification:** After passing through the encoder, the output vector corresponding to the '[CLS]' token's position is extracted. This vector, representing the entire image, is then passed through a simple Multi-Layer Perceptron (MLP) for the final image classification prediction.

This pipeline effectively transforms an image into a "language" that the powerful Transformer architecture can understand and process.

ViT vs. CNNs: A Paradigm Shift

ViTs and CNNs approach vision tasks from fundamentally different perspectives.

Convolutional Neural Networks (CNNs)

- **Strong Inductive Bias:** CNNs have built-in assumptions about images.
  - *Locality:* Filters process local neighborhoods.
  - *Translation Equivariance:* An object is recognized regardless of its position.

ViT vs. CNNs: A Paradigm Shift

ViTs and CNNs approach vision tasks from fundamentally different perspectives.

Convolutional Neural Networks (CNNs)

- **Strong Inductive Bias:** CNNs have built-in assumptions about images.
  - *Locality:* Filters process local neighborhoods.
  - *Translation Equivariance:* An object is recognized regardless of its position.
- **Data Efficiency:** These biases make CNNs very data-efficient and powerful on small to medium-sized datasets.

Vision Transformers (ViTs)

- **Weak Inductive Bias:** ViTs make almost no prior assumptions about the image structure, aside from the initial patching.

ViT vs. CNNs: A Paradigm Shift

ViTs and CNNs approach vision tasks from fundamentally different perspectives.

Convolutional Neural Networks (CNNs)

- **Strong Inductive Bias:** CNNs have built-in assumptions about images.
  - *Locality:* Filters process local neighborhoods.
  - *Translation Equivariance:* An object is recognized regardless of its position.
- **Data Efficiency:** These biases make CNNs very data-efficient and powerful on small to medium-sized datasets.

Vision Transformers (ViTs)

- **Weak Inductive Bias:** ViTs make almost no prior assumptions about the image structure, aside from the initial patching.
- **Global Relationships:** Self-attention allows the model to learn relationships between distant patches.

ViT vs. CNNs: A Paradigm Shift

ViTs and CNNs approach vision tasks from fundamentally different perspectives.

Convolutional Neural Networks (CNNs)

- **Strong Inductive Bias:** CNNs have built-in assumptions about images.
  - *Locality:* Filters process local neighborhoods.
  - *Translation Equivariance:* An object is recognized regardless of its position.
- **Data Efficiency:** These biases make CNNs very data-efficient and powerful on small to medium-sized datasets.

Vision Transformers (ViTs)

- **Weak Inductive Bias:** ViTs make almost no prior assumptions about the image structure, aside from the initial patching.
- **Global Relationships:** Self-attention allows the model to learn relationships between distant patches.
- **Data Hungry:** Due to the lack of built-in biases, ViTs require massive datasets (e.g., JFT-300M, ImageNet-21k) during pre-training.

ViT vs. CNNs: A Paradigm Shift

ViTs and CNNs approach vision tasks from fundamentally different perspectives.

Convolutional Neural Networks (CNNs)

- **Strong Inductive Bias:** CNNs have built-in assumptions about images.
  - *Locality:* Filters process local neighborhoods.
  - *Translation Equivariance:* An object is recognized regardless of its position.
- **Data Efficiency:** These biases make CNNs very data-efficient and powerful on small to medium-sized datasets.

Vision Transformers (ViTs)

- **Weak Inductive Bias:** ViTs make almost no prior assumptions about the image structure, aside from the initial patching.
- **Global Relationships:** Self-attention allows the model to learn relationships between distant patches.
- **Data Hungry:** Due to the lack of built-in biases, ViTs require massive datasets (e.g., JFT-300M, ImageNet-21k) during pre-training.

When pre-trained on enormous datasets, **ViTs often outperform state-of-the-art CNNs** when fine-tuned on downstream tasks.